LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK AND QUEUE

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin **NIM. 2010817110011**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....i
DAFTAR ISI.....ii
DAFTAR GAMBAR.....iii
DAFTAR TABEL.....iv
SOAL 1.....1
A. Source Code.....1
B. Pembahasan.....2
SOAL 2.....4
A. Source Code.....5
B. Output Program.....6
C. Pembahasan.....7
SOAL 3.....9
A. Source Code.....9
B. Pembahasan.....10
SOAL 4.....13
A. Source Code.....14
B. Output Program.....15
C. Pembahasan.....15
TAUTAN GIT.....17

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2.....7
Gambar 2. Screenshot Hasil Jawaban Soal 2.....15

DAFTAR TABEL

Table 1. Source Code stack.cpp.....2
Table 2. Contoh input dan output soal 2.....5
Table 3. Source Code prob1.cpp.....6
Table 4. Source Code queue.cpp.....10
Table 5. Contoh input dan output soal 4.....14
Table 6. Source Code prob2.cpp.....15

SOAL 1

Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

stack.cpp

```
#include "stack.h"

void init(Stack *s) {
  s->top = new int;
  *s->top = -1;
}
bool isEmpty(const Stack *s) { return *s->top <= -1; }
bool isFull(const Stack *s) { return *s->top >= MAX - 1;
}
void push(Stack *s, int value) {
  if (isFull(s)) {
    throw "Stack Penuh";
  }
  *(s->top) += 1;
  s->data[*s->top] = value;
}
void pop(Stack *s) {
  if (isEmpty(s)) {
    throw "Stack Kosong";
  }
  *(s->top) -= 1;
}
int peek(const Stack *s) {
  if (isEmpty(s)) {
    throw "Stack Kosong";
  }
  return s->data[*s->top];
```

```
}
```

Table 1. Source Code stack.cpp

B. Pembahasan

Pada soal ini ada 6 fungsi yang perlu di implementasikan ada 1 yang berfungsi seperti constructor, 3 sebagai getter yang hanya melakukan cek / return sesuatu, dan 2 fungsi utama operasi.

Alur Fungsi

- init() : Pertama fungsi akan melakukan inisialisasi untuk field top biar nilai nya bisa di set, Kedua fungsi ini akan memberikan nilai -1 sebagai nilai awal yang menandakan stack ini kosong.
- isEmpty() : Fungsi ini hanya melakukan cek apakah nilai dari top kurang atau sama dengan -1, untuk menandakan stack ini kosong.
- isFull() : Fungsi ini hanya melakukan cek apakah nilai dari top lebih atau sama dengan MAX dikurang 1 yang ada di header file.
- peek() : Pertama fungsi ini akan memanggil fungsi isEmpty untuk cek apakah stack kosong, jika kosong langsung throw kosong saja. Kedua fungsi ini akan mengembalikan nilai stack yang paling atas.
- push() : Pertama fungsi ini akan memanggil fungsi isFull untuk cek apakah stack penuh. Jika stack penuh akan throw penuh. Kedua fungsi ini akan menambah 1 nilai dari pointer top atau memajukan nya. Ketiga dia akan memasukan nilai baru kedalam posisi top yang baru.
- pop() : Pertama fungsi ini akan memanggil fungsi isEmpty untuk cek apakah stack kosong, jika kosong langsung throw kosong saja. Kedua fungsi ini akan mengurangi satu nilai dari top, atau menurunkan nilainya sehingga posisi top bergeser kebawah.

Kenapa Ini Bekerja?

Init() dibuat sebagai constructor dimana tugas utama dari constructor adalah melakukan initialisasi dari sebuah object pada kasus ini adalah stack. Maka dari itu fields atau atribut nya yang pada header berbentuk pointer perlu di inisialisasi juga supaya memiliki alamat pointer, jika tidak pointer itu tidak akan memiliki alamat memory sehingga nilai nya tidak bisa dimasukan.

IsEmpty dibuat untuk cek apakah stack ini kosong, dan karena kondisi awal stack saat kosong top nya diset ke -1 maka kita hanya perlu cek apakah top stack saat ini kurang atau sama dengan -1.

isFull dibuat untuk cek apakah stack ini penuh, karena max sudah ada didefenisikan pada file header kita hanya perlu lakukan cek apakah top sama dengan max – 1 karena index nya mulai dari 0.

peek dibuat untuk mengembalikan nilai teratas / terbaru dari stack, untuk memastikan fungsi ini berjalan dengan sempurna kita perlu cek apakah stack kosong dulu, untuk mengembalikan nilai stack nya kita cukup pangil saja data nya dengan index top.

push dibuat untuk menambahkan data kedalam stack, sebelum fungsi ini bisa menambahkan data kedalam stack dia perlu cek apakah stack penuh dulu biar tidak terjadi overflow, fungsi ini akan increment nilai top baru memasukan data nya kedalam stack dengan index top, karena ada cek ini kita tidak bisa memasukan nilai yang lebih besar dari max.

pop dibuat untuk mengeluarkan data dari dalam stack, karena data dalam stack tidak perlu dihapus dan bisa di overwrite maka fungsi ini tidak perlu melakukan reset atau null kan data, tetapi hanya perlu menurunkan nilai top saja karena kita hanya perlu data yang ada dibawah top dan top saja, dan jika setiap pop data sebelumnya di hapus maka membuat fungsi ini jauh lebih lambat.

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu O(1), karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n.

SOAL 2

Si Palui dan Mesin Hitung Aneh

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

Table 2. Contoh input dan output soal 2

A. Source Code

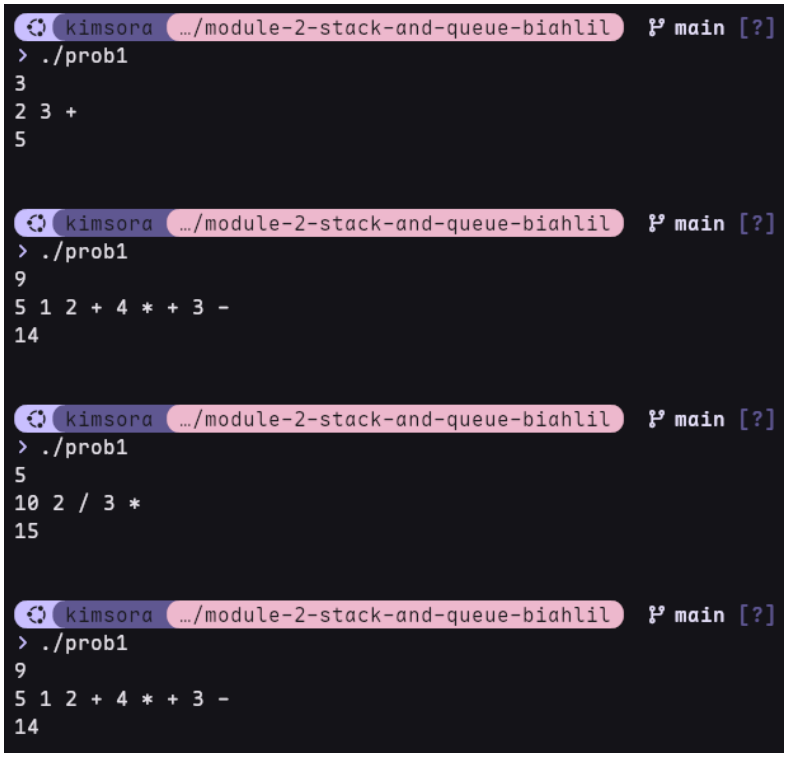
prob1.cpp

```
#include "stack.h"
#include <cmath>
#include <iostream>
#include <string>
using namespace std;

// Postfix eval
int main() {
  // init
  int n;
  Stack stack1;
  init(&stack1);

  // input n
  cin >> n;
  //   loop input sebanyak n
  for (int i = 0; i < n; i++) {
     // container untuk input pake string karena campur
symbol
    string input;
    cin >> input;

```

```
     // cek apakah input digit atau jika size nya lebih
dari 1 dan memiliki -
    // intinya cek jika ini bukan symbol
    if (isdigit(input[0]) || input.size() > 1 && input[0]
== '-') {
      // push ke stack
      push(&stack1, stoi(input));
    } else {
      // jika bukan symbol keluarkan 2 angka
      int kanan = peek(&stack1);
      pop(&stack1);
      int kiri = peek(&stack1);
      pop(&stack1);

      //   lakukan proses seuai dengan symbol
      if (input == "+")
        push(&stack1, kiri + kanan);
      else if (input == "-")
        push(&stack1, kiri - kanan);
      else if (input == "*")
        push(&stack1, kiri * kanan);
      else if (input == "/")
        push(&stack1, kiri / kanan);
      else if (input == "^")
        push(&stack1, (int)pow(kiri, kanan));
    }
  }
  cout << peek(&stack1) << endl;
  return 0;
}
```

Table 3. Source Code prob1.cpp

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

- init : ini adalah bagian untuk menginisialisasi variable dan struct yang diperlukan, pada algoritma ini adalah variable n, stack stack1.
- Input n : mengambil input n dari user.
- Loop : disini kita melakukan loop sebanyak n kali untuk melakukan postfix eval.
- User input : didalam loop ini inisialisasi variable bantuan “input” untuk menampung input user, dan kemudian mengambil input dari user untuk “input”.
- Check if not symbol : masih didalam loop, kita lakukan cek apakah input user ini adalah angka bulat dan bukan symbol, jika cek ini benar maka masukan angka ke stack1
- If symbol : jika symbol maka kita perlu mengeluarkan 2 angka terakhir dari stack1 dengan cara inisialisasi 2 variable bantuan kanan dan kiri, masukan angka terbaru

(atas) kedalam kanan dan lakukan pop pada stack1, dan kemudian masukan angka kedua (setelah atas) ke kiri dan lakukan pop pada stack1.

- Process sysmbol : setelah kedua angka kanan dan kiri dikeluarkan maka algoritma melakukan cek sysmbol apa yang sesuai dengan input, dan lakukan operasi arithmatic nya masing-masing, dan hasil nya akan di push kedalam stack1.
- Hal ini dilakukan terus sampai n kali.
- Output : Print isi terakhir yang ada pada stack1 yang merupakan hasil dari proses.

Kenapa Ini Bekerja?

Algoritma ini bekerja dengan cara mengambil n input dari user dan melakukan postfix eval sebanyak n kali. Karena sifat stack yang Last In, First Out (LIFO) membuatnya sangat cocok untuk postfix karena sifat ini memastikan symbol yang muncul akan selalu diproses terhadap 2 angkat teratas. Dan hasilnya akan disimpan kembali kedalam stack, yang kemudian proses nya bisa dilanjutkan, jika yang masuk setelah ini angka maka simpan, dan jika yang masuk symbol maka angka hasil proses sebelum nya dan yang dibawah nya akan kembali di proses, hingga hanya ada sisa satu angka yang ada pada stack.

Analisis Kompleksitas

Algoritma ini memiliki kompleksitas waktu dan ruang O(n), karena algoritma ini memiliki 1 loop sebanyak n kali yang membuat waktu nya menjadi O(n), dan algoritma ini juga membuat sebuah struktur data yang size nya sebesar n hal ini membuat kompositas ruang nya menjadi O(n).

SOAL 3

Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

queue.cpp

```
#include "queue.h"

void init(Queue *q) {
  q->front = new int;
  q->rear = new int;
  *q->front = -1;
  *q->rear = -1;
}
bool isEmpty(const Queue *q) { return *q->front == -1; }
bool isFull(const Queue *q) { return (*q->rear + 1) % MAX
== *q->front; }
void enqueue(Queue *q, int value) {
  if (isFull(q)) {
    throw "Queue Penuh";
  }
  if (isEmpty(q)) {
    *q->front = 0;
    *q->rear = 0;
  } else {
    *q->rear = (*q->rear + 1) % MAX;
  }
  q->data[*q->rear] = value;
}
void dequeue(Queue *q) {
  if (isEmpty(q)) {
    throw "Queue Kosong";
  }
  if (*q->front == *q->rear) {
```

```
    *q->front = -1; // Reset ke setelan pabrik
    *q->rear = -1;
  } else {
    // Jika masih banyak elemen, front maju melingkar
    *q->front = (*q->front + 1) % MAX;
  }
}
int front(const Queue *q) {
  if (isEmpty(q)) {
    throw "Queue Kosong";
  }
  return q->data[*q->front];
}
int back(const Queue *q) {
  if (isEmpty(q)) {
    throw "Queue Kosong";
  }
  return q->data[*q->rear];
}
```

Table 4. Source Code queue.cpp

B. Pembahasan

Pada soal ini ada 7 fungsi, dimana 1 fungsi constructor, 4 getter, dan 2 fungsi utama.

Alur Fungsi

- Init() : hampir sama dengan di soal 1 yang membedakan disini inisialisasi 2 field yaitu front dan rear.
- IsEmpty() : Fungsi ini melakukan cek apakah queue kosong dengan cara cek apakah front sama dengan -1.
- isFull() : Fungsi ini melakukan cek apakah queue penuh dengan cara membandingkan apakah rear + 1 mod Max sama dengan front, dimana ini cek apakah queue berikutnya memiliki posisi sama dengan front menandakan queue penuh dan tidak ada posisi kosong.
- front() : Fungsi ini pertama memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua return data queue dengan index front.
- back() : Fungsi ini pertama memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua return data queue dengan index rear.
- enqueue() : Pertama fungsi ini memanggil isFull, jika full throw queue penuh. Kedua fungsi ini memanggil isEmpty, jika kosong fungsi ini akan mengubah front dan rear

menjadi 0 biar siap untuk di masukan data, jika tidak kosong fungsi ini akan memajukan rear dengan rear + 1 mod Max, supaya jika rear sudah posisi max maka akan kembali ke 0 lagi untuk mengisi posisi yang kosong.

- dequeue() : Pertama fungsi ini memanggil isEmpty untuk cek apakah queue kosong, jika kosong throw queue kosong. Kedua fungsi ini melakukan cek apakah front dan rear memiliki posisi yang sama, reset front dan rear menjadi -1 untuk menandakan queue kosong, jika tidak front + 1 mod max artinya front akan maju satu posisi dan jika sudah mencapai max front akan kembali ke posisi 0.

Kenapa Ini Bekerja?

Init sama seperti soal 1 hanya saja bedanya disini ada 2 field yang perlu di inisialisasi.

IsEmpty adalah fungsi getter yang bertugas untuk cek apakah queue kosong, fungsi melakukan cek apakah queue berada pada kondisi default atau kosong nya yang ditandai dengan front dan rear memiliki nilai -1, ini diset pada init dan saat dequeue menemukan front dan rear nya pada posisi yang sama artinya queue kosong dia yang akan set front dan rear menjadi -1

isFull adalah fungsi getter yang bertugas untuk cek apakah queue penuh, fungsi ini melakukan cek dengan cara membandingkan apakah posisi rear + 1 mod max (posisi rear setelah ini) sama dengan front, yang menandakan posisi rear setelah ini sama dengan front dan ini indikasi bahwa queue penuh. Karena rear maju ketemu front itu artinya penuh.

Front adalah fungsi getter yang bertugas untuk mengembalikan data yang ada pada front, fungsi ini melakukannya dengan mengembalikan data pada posisi front. Fungsi ini melakukan cek isEmpty untuk memastikan queue tidak kosong.

Rear adalah fungsi getter yang bertugas untuk mengembalikan data yang ada pada rear, hal ini dilakukan dengan mengembalikan data yang ada pada posisi rear. Fungsi ini melakukan cek isEmpty untuk memastikan queue tidak kosong.

Enqueue fungsi ini adalah fungsi utama yang bertugas untuk memasukan data kedalam queue, sebelum memasukan data fungsi ini melakukan cek isFull untuk memastikan queue tidak penuh dan data bisa dimasukan, kemudian fungsi ini melakukan cek isEmpty, hal ini dilakukan untuk memastikan queue sudah siap dan field nya tidak kosong, jika kosong fungsi ini akan menyiapkan queue dengan mengubah front dan rear menjadi 0, jika queue tidak kosong maka fungsi akan memajukan posisi rear tentu saja dengan mod Max, dan kemudian fungsi akan memasukan data ke posisi rear. Bisa terlihat fungsi ini cukup ketat dengan pengecekan nya, hal ini untuk mencegah ada nya overlap antara data yang ada di rear dan front.

Dequeue fungsi adalah fungsi utama yang bertugas untuk mengeluarkan data dari queue yaitu data di front, sebelum fungsi ini memajukan front fungsi ini melakukan isEmpty, dan

melakukan cek apakah posisi front dan rear sama, jika sama artinya queue ini sisa satu posisi dan kita akan reset kondisi nya ke -1 yaitu kosong, jika tidak maka fingsi ini akan memajukan posisi front tentu saja dengan mod Max supaya posisi nya bisa kembali ke 0 jika sudah max. fungsi ini lah yang bekerja untuk menjaga status queue karena dia yang bisa melakukan reset queue menjadi kosong kembali

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu O(1), karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n.

SOAL 4

Si Palui dan Mesin Hitung Aneh

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Input	**Output**
5 3 1 2 3 4 5	6 9 12

5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

Table 5. Contoh input dan output soal 4

A. Source Code

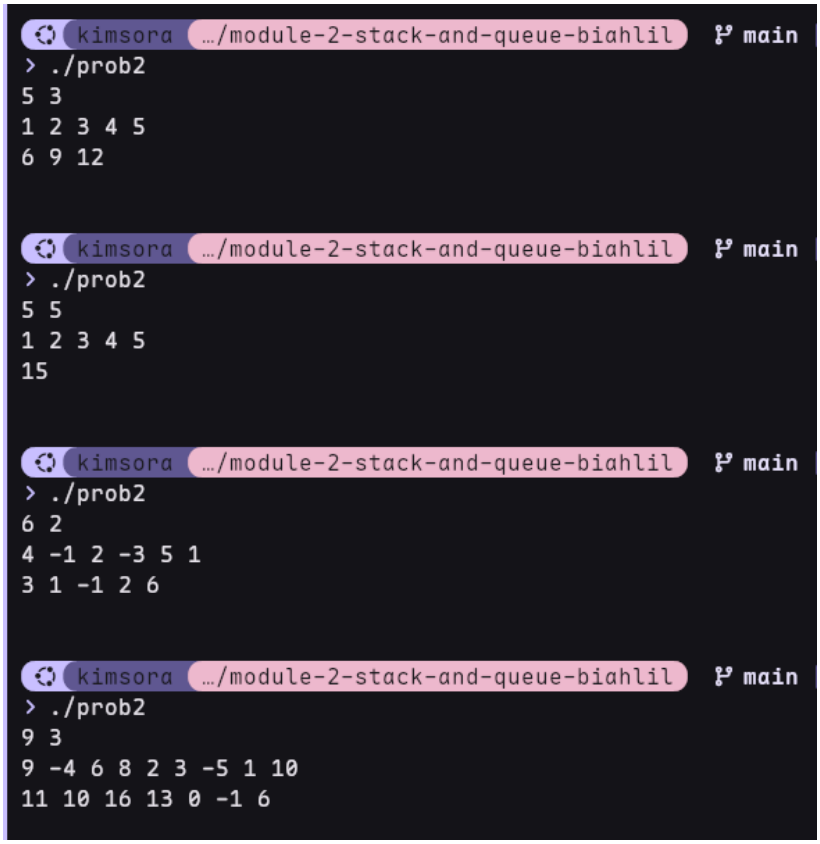
prob2.cpp

```
#include "queue.h"
#include <iostream>
using namespace std;

// sliding window queue
int main() {
  // init
  int n;
  int k;
  int total = 0;
  Queue queue1;
  init(&queue1);

  //   input n dan k (n = banyak input, k = window yang
ingin dijumlahkan)
  cin >> n >> k;
  for (int i = 0; i < n; i++) {
    // container input angka
    int input;
    cin >> input;
    //  lanjut isi jika queue nya belum penuh alias belum
sepanjang k
    if (i < k) {
      enqueue(&queue1, input);
      total += input;
      //   jika sudah sepanjang k print total
      if (i == k - 1) {
        cout << total;
      }
    } else {
      // untuk lebihan akan pake sliding window
      int yang_dibuang = front(&queue1);
      dequeue(&queue1);
      enqueue(&queue1, input);
       //   total cuman mengurangkan nilai yang dibuang
```

```
alias front dna ditambah
      //   nilai baru alias rear baru.
      total = total - yang_dibuang + input;
      cout << " " << total;
    }
  }
  cout << endl;
  return 0;
}
```

Table 6. Source Code prob2.cpp

B. Output Program



Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

- init : ini adalah bagian untuk menginisialisasi variable dan struct yang diperlukan, pada algoritma ini adalah variable n k total, queue queue1.
- Input n dan k : mengambil input n dan k dari user.
- Loop : disini kita melakukan loop sebanyak n kali untuk melakukan sliding window sum.
- User input : didalam loop ini inisialisasi variable bantuan “input” untuk menampung input user, dan kemudian mengambil input dari user untuk “input”.
- Check if i < k : masih didalam loop ini algoritma akan melakukan cek apakah k atau window sudah penuh, jika belum maka input user akan di enqueue. Dan kemudian tambahkan nilai input user kedalam total. Masih didalam cek window dilakukan cek i == k – 1 jika input sudah memenuhi window maka bisa print total.
- Lebihan sliding window : jika check window sudah penuh maka kita perlu inisialisasi variable bantuan yang_dibuang yang memiliki value dari data yang berada pada posisi front queue. Kemudian algoritma akan melakukan dequeue yang dilanjutkan dengan enqueue input terbaru user.
- Total baru : setelah dequeue dan enqueue dilakukan maka algoritma akan memperbarui nilai dari total dengan dikurangi oleh yang_dibuang dan ditambah oleh input terbaru. Dan kemudian total akan di print.
- Hal ini dilakukan terus sampai n kali.

Kenapa Ini Bekerja?

Algoritma ini bekerja dengan cara mengambil input n dan k dari user, dan melakukan proses sliding window untuk mengambil total nilai dari setiap window. Sifat First in First out dari queue sangat cocok untuk menyelsaikan masalah pada soal ini, dan keperluan untuk melakukan operasi untuk beberapa member queue saja membuat algoritma sliding window ini menjadi solusi yang pass. Algoritma ini bekerja dengan cara mengisi window / member yang ingin diproses, dan jika sudah penuh baru lakukan proses, ini dilakukan seterus nya seiring dengan memberlama dikeluarkan dan member baru dimasukan.

Analisis Kompleksitas

Algoritma ini memiliki kompleksitas waktu O(n) dan Kompleksitas Ruang O(k), karena algoritma ini memiliki 1 loop sebanyak n kali yang membuat waktu nya menjadi O(n), dan untuk kompleksitas ruang, nilainya adalah O(k), karena meskipun total data ada sebanyak n, struktur data Queue pada algortima ini hanya akan menyimpan elemen maksimal sebanyak kapasitas jendela (k) di memori secara bersamaan (karena elemen lama langsung dikeluarkan).

TAUTAN GIT

Repo lengkap : https://github.com/biahlil/Algoritma-dan-Struktur-Data.git
Stack and Queue: https://github.com/DSA26-ULM/module-2-stack-and-queue-biahlil.git